
Building and Analysing a Near-Real-Time Data Warehouse

Project Report

DS3004 Data Warehousing and Business Intelligence
Project

Submitted to
Dr. Muhammad Asif Naeem

Submitted by
Fatima Tu Zahra 23I-2616

FAST National University of Computing and Emerging Sciences, Islamabad

Building and Analysing a Near-Real-Time Data Warehouse.....	3
Project Overview.....	3
Data Source.....	3
Extraction.....	3
Transform.....	3
Loading.....	3
Schema.....	3
Original Schema.....	4
Adjusted Schema.....	5
Slowly Changing Dimensions.....	6
HybridJoin.....	7
Algorithm.....	7
Three Shortcomings.....	8
Summary of Lessons.....	8
Importance of Schema Design in Data Warehousing.....	8
Handling Imperfect Data Sources.....	8
Practical vs. Theoretical Data Warehousing.....	9
References.....	9
Outputs.....	10

Building and Analysing a Near-Real-Time Data Warehouse

Project Overview

The goal of this project was to design, implement, and analyze a near-real-time data warehouse for Walmart.

Data Source

The provided data sources were three CSV files, containing data generated by customers.

Extraction

The data was extracted in **micro batches** from the CSV files and passed through the ETL pipeline, which then loaded it onto the MySQL data warehouse.

Transform

Columns for day, month, year, quarter, and is_weekend were added through feature engineering on the data column.

The data type for the primary key of products was changed from string to integer to make indexing more useful when selecting disk partitions. `P010042`, for example, was changed to `10042`. Additionally, it was noted that the two digits `42` appeared at the end of every primary key, and removing them would not cause duplicate rows, so we further converted it to `100` by stripping away the last two digits. This made disk partitions more logical and efficient.

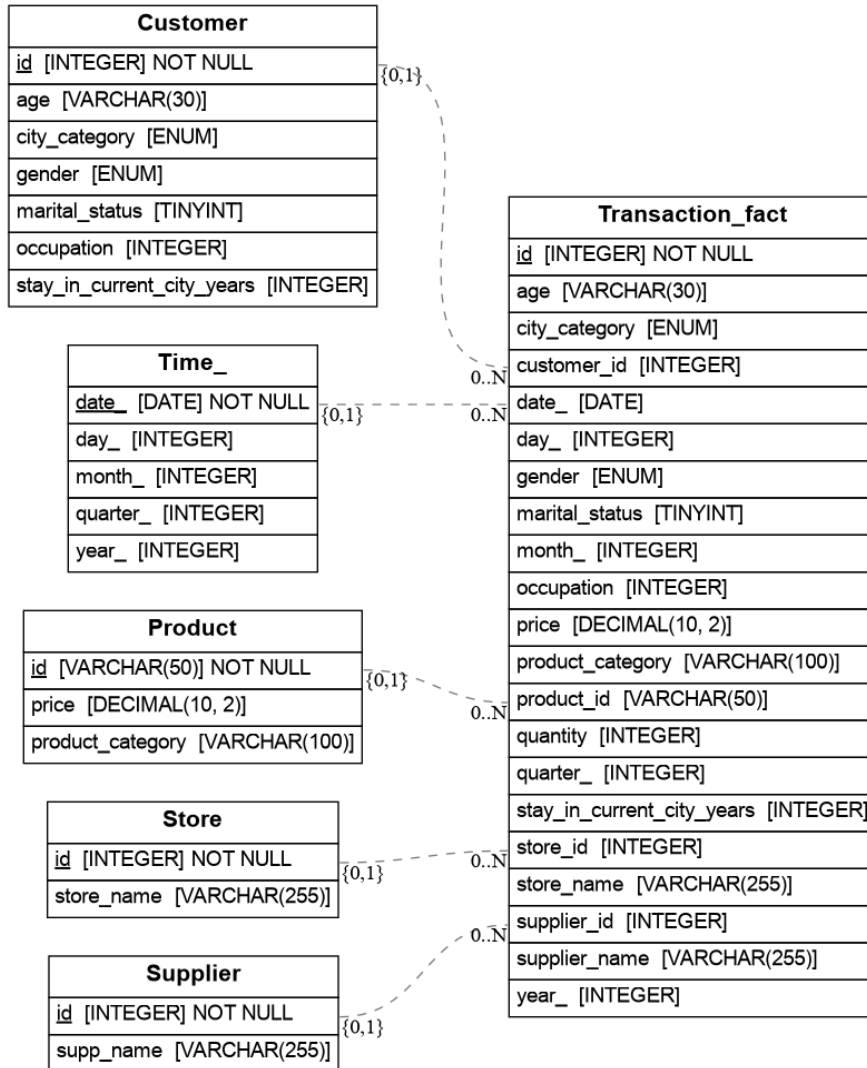
Loading

The data warehouse was implemented using a MySQL database. A star schema was implemented, consisting of 4 dimension tables and a central fact table. First, the master data is loaded from the data source into the data warehouse and kept updated using Slowly Changing Dimension type 2.

Schema

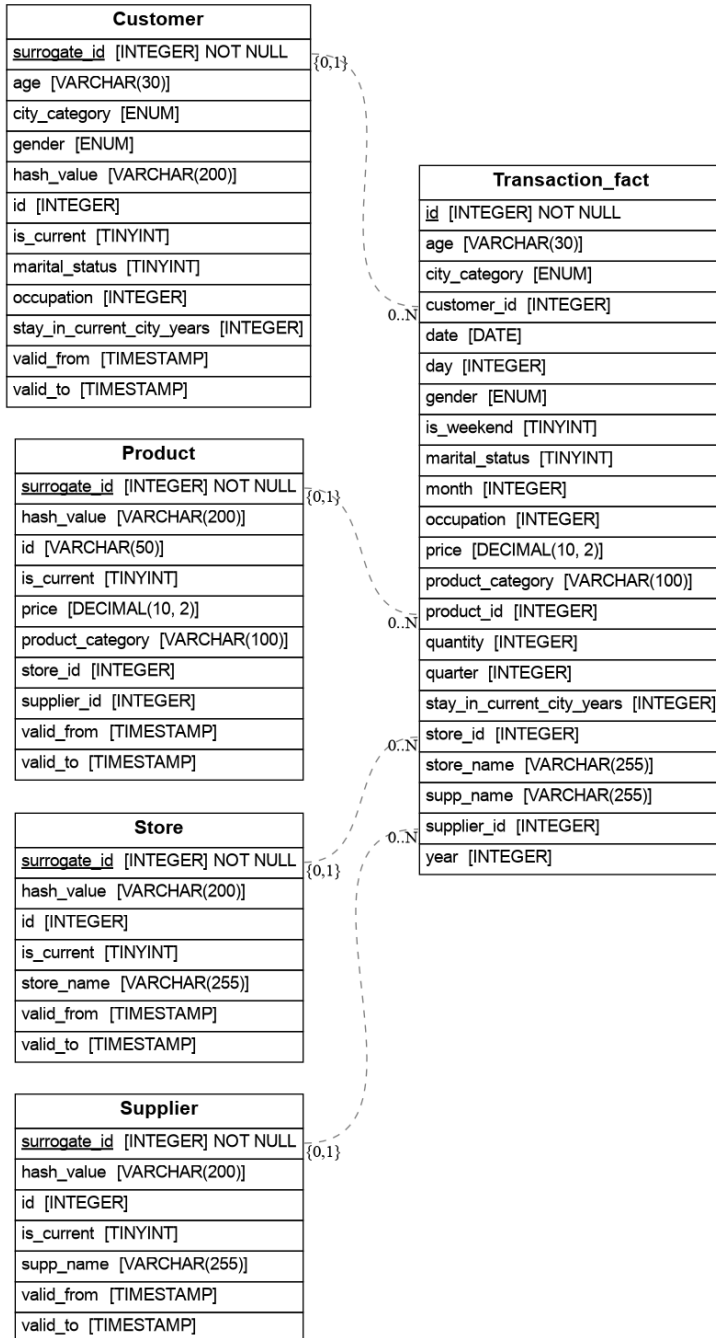
A star schema was required for the data warehouse; two versions were made, but due to some problems in the source data, we had to make adjustments.

Original Schema



Here, we had five-dimensional tables: Customer, Product, Time, Store, and Supplier. The table was normalized, and it was ensured that dimension tables made it easy to implement slowly changing dimensions.

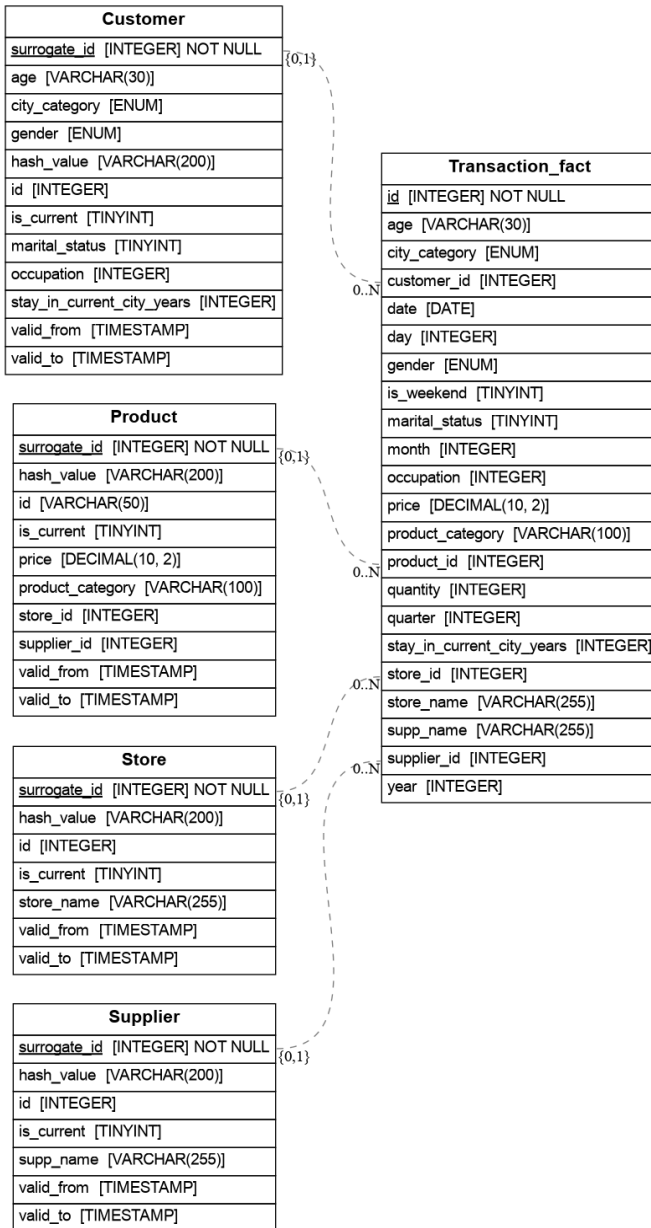
Adjusted Schema



The provided source data did not contain separate data for the time dimension. Nor was the transaction data connected directly to the Store and Supplier data.

To go by the rules, we would have to connect the Store and Supplier table to the product dimension table first, before connecting the product to the transaction fact table. This was not done as then it would be a snowflake schema, not the required star schema.

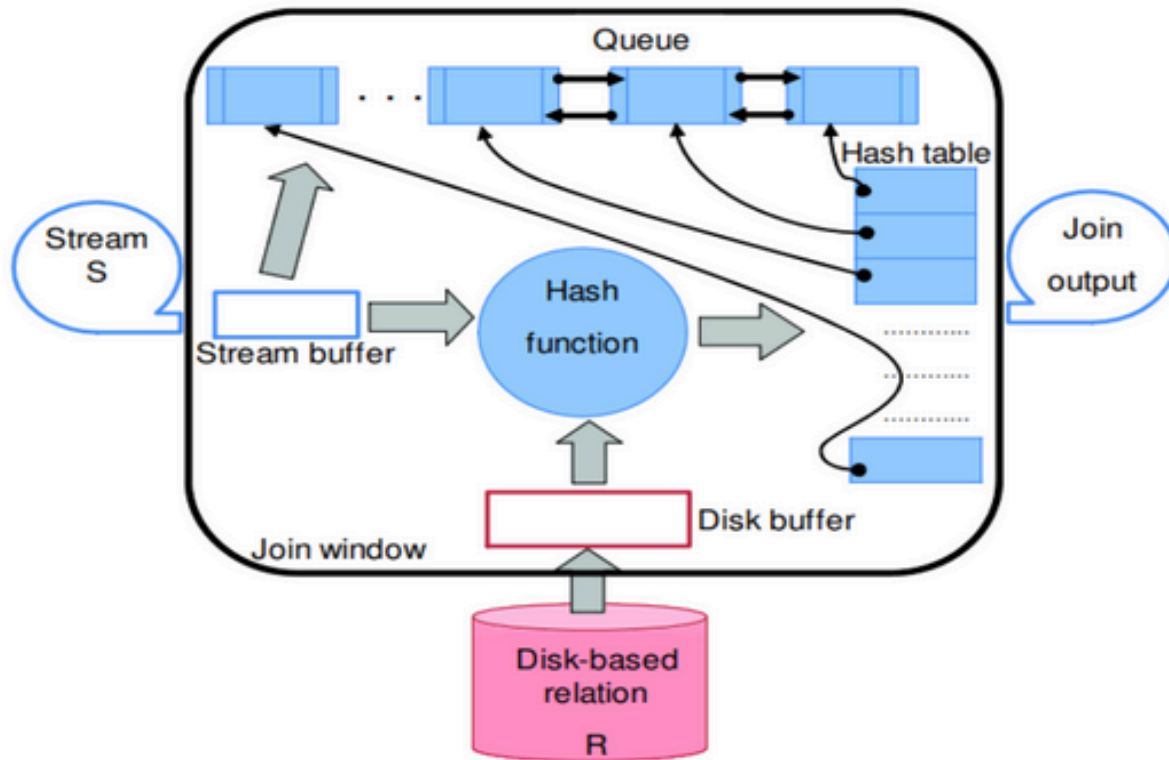
Slowly Changing Dimensions



Slowly changing dimension, type 2 was used for ‘As Is’, ‘As Was’ query support.

A surrogate key had to be used to support multiple rows with the same attribute and historical data. The original key was used as a business key. A hash function was used to detect a change in rows, and the rows whose hash did not match were updated.

HybridJoin



Algorithm

1. Initialize:
 - an empty Queue Q
 - an empty Hash Map H
 - an empty Disk Buffer D
 - an empty Stream Buffer S
2. Continuously read streaming data into Stream Buffer S.
3. For each incoming tuple `t` in S:
 - a. Extract the join key `k` from t.
 - b. Enqueue `k` into Queue Q.
 - c. Insert (or append) t into Hash Map H using:
 $H[k] \rightarrow [t_1, t_2, \dots, t_n]$ # support duplicates

4. While the Hash Map H is not empty:
 - a. Identify the oldest join key in Q; let's call it 'k_oldest'.
 - b. Load from disk into Disk Buffer D the dimension table partition that contains 'k_oldest'.
 - c. For each record 'd' in Disk Buffer D:
 - i. Let 'k_dim' be the join key of d.
 - ii. If H contains 'k_dim':
 1. Retrieve all matching stream tuples from H[k_dim].
 2. Join each stream tuple in H[k_dim] with d.
 3. Append each joined result to the output.
 4. Remove all occurrences of 'k_dim' from Queue Q.
 5. Delete H[k_dim] from Hash Map H.
5. If the Disk Buffer D is exhausted and Hash Map H is not empty:
 - a. Load the next disk partition using the next oldest join key from Q.
 - b. Repeat Step 4.
6. When the hash map H becomes empty:
 - a. All stream tuples have been processed and joined.
 - b. Terminate the algorithm.

Cascading Joins were used to join the stream to each dimension table.

Three Shortcomings

1. It uses a single disk buffer. If the relevant keys are not found in it, we have to wait for the next partition to be loaded from the disk all over again. If it had multiple disk buffers, we could check them one by one, keeping the time complexity the same instead of overwriting them.
2. Only the oldest keys in the queue are prioritized instead of a more heuristic approach. Oldest keys in the queue may be problematic, and it might reduce the effectiveness of a near-real-time data warehouse if it insists on the data warehouse being filled in the right order.
3. Join Operation is blocked while the disk buffer is loading; it does not parallelize the process.

Summary of Lessons

Importance of Schema Design in Data Warehousing

This project taught me how pivotal good schema design is; it took many different versions and wasted hours of coding when I realized I had to change it only after having implemented it halfway.

Handling Imperfect Data Sources

The data sources in this scenario were not perfect. Seeing the ETL pipeline transform that into something usable has shown the power of data transformation and how imperative it is to the insights one can get from the data.

Practical vs. Theoretical Data Warehousing

In a completely theoretical scenario, perhaps we may have done the schema or the join differently, but due to real-life constraints and facts, a lot of compromises have to be made. Not everything can be perfectly normalized.

References

1. Naeem, M. A., Dobbie, G., & Weber, G. (2011). HYBRIDJOIN for Near-Real-Time Data Warehousing. *International Journal of Data Warehousing and Mining (IJDWM)*, 7(4), 21-42.
<https://doi.org/10.4018/jdwm.2011100102>
2. IEEE Xplore Full-Text PDF: (n.d.).
<https://ieeexplore.ieee.org/stamp/stamp.jsp?tp=&arnumber=9371671>

Outputs

**Note: At the time these queries ran, the warehouse was not fully populated; only 53 000 rows were inserted

1

Results 1 ×						
(SELECT tf.product_id, SUM(tf.price * tf.quantity) AS revenue, ⚙ Enter a SQL expression to filter results (use C						
Grid	123 product_id	123 revenue	123 is_weekend	123 month	123 year	
1	639	332.15	0	5	2,020	
2	1,104	271.4	0	10	2,020	
3	1,366	238.23	0	4	2,020	
4	539	233.19	0	1	2,020	
5	763	230.55	0	10	2,020	
6	1,642	239.79	1	1	2,020	
7	1,121	228.87	1	9	2,020	
8	730	220.17	1	8	2,020	
9	276	211.62	1	4	2,020	
10	137	200.52	1	5	2,020	

2

Results 1 ×				
SELECT tf.city_category, tf.gender tf.age, SUM(tf.price * tf.qua ⚙ Enter a SQL expression to filter results (use C				
Grid	AZ city_category	AZ gender	AZ age	123 purchase_amount
1	A	M	0-17	360.13
2	A	M	18-25	7,689.23
3	A	M	26-35	11,949.02
4	A	M	36-45	4,267.97
5	A	M	46-50	1,786.54
6	A	M	51-55	980.28
7	A	M	55+	667.27
8	A	M	[NULL]	27,700.44
9	A	F	0-17	74.64
10	A	F	18-25	1,206.45
11	A	F	26-35	2,053.38
12	A	F	36-45	1,493.55

Results 1 ×

 SELECT tf.product_category, tf.occupation, SUM(tf.quantity) AS  Enter a SQL expression

 Grid		AZ product_category ▼	123 occupation ▼	123 total_sales ▼	
 Text	1	Appliances	4	3	
	2	Appliances	6	3	
	3	Appliances	7	2	
	4	Appliances	12	2	
	5	Appliances	16	1	
	6	Appliances	17	2	
	7	Appliances	20	4	
	8	Appliances	[NULL]	17	
	9	Arts, Crafts & Sewing	0	4	
	10	Arts, Crafts & Sewing	2	2	
	11	Arts, Crafts & Sewing	3	4	
	12	Arts, Crafts & Sewing	6	8	
	13	Arts, Crafts & Sewing	9	2	
	14	Arts, Crafts & Sewing	10	3	
	15	Arts, Crafts & Sewing	11	1	
	16	Arts, Crafts & Sewing	14	1	
	17	Arts, Crafts & Sewing	16	4	
	18	Arts, Crafts & Sewing	17	1	
	19	Arts, Crafts & Sewing	20	1	
	20	Arts, Crafts & Sewing	[NULL]	31	
 Record	21	Automotive	0	5	
	22	Automotive	1	8	
	23	Automotive	2	2	
	24	Automotive	3	5	
	25	Automotive	4	4	
	26	Automotive	6	2	
	27	Automotive	7	12	
	28	Automotive	9	2	
	29	Automotive	10	3	
	30	Automotive	11	3	
	31	Automotive	12	4	
	32	Automotive	15	2	
	33	Automotive	16	3	

Results 1 ×					
SQL SELECT tf.quarter, tf.gender, tf.age, SUM(tf.price * tf.quantity) Enter a SQL expression					
	123 quarter	AZ gender	AZ age	123 total_purchase	
1	1	M	0-17	1,225.87	
2	1	M	18-25	9,231.88	
3	1	M	26-35	18,076.61	
4	1	M	36-45	7,202.37	
5	1	M	46-50	3,510.18	
6	1	M	51-55	2,484.01	
7	1	M	55+	1,036.13	
8	1	M	[NULL]	42,767.05	
9	1	F	0-17	337.92	
10	1	F	18-25	2,422.59	
11	1	F	26-35	4,072.57	
12	1	F	36-45	2,405.86	
13	1	F	46-50	1,584.37	
14	1	F	51-55	1,072.89	
15	1	F	55+	726.91	
16	1	F	[NULL]	12,623.11	
17	1	[NULL]	[NULL]	55,390.16	
18	2	M	0-17	855.79	
19	2	M	18-25	9,575.68	
20	2	M	26-35	12,868.61	
21	2	M	36-45	7,758.02	
22	2	M	46-50	2,895.55	
23	2	M	51-55	3,066.75	
24	2	M	55+	1,057.31	
25	2	M	[NULL]	38,077.71	
26	2	F	0-17	394.56	
27	2	F	18-25	2,458.14	
28	2	F	26-35	5,062.09	
29	2	F	36-45	1,986.35	
30	2	F	46-50	826.11	
31	2	F	51-55	1,226.63	
32	2	F	55+	471.37	

Results 1

SELECT tf.product_category, tf.occupation, SUM(tf.price * tf.q) Enter a SQL expression

	AZ product_category	123 occupation	123 total_sales
1	Appliances	4	84.45
2	Appliances	6	143.52
3	Appliances	7	10.22
4	Appliances	12	56.9
5	Appliances	16	3.6
6	Appliances	17	56.9
7	Appliances	20	81.22
8	Appliances	[NULL]	436.81
9	Arts, Crafts & Sewing	0	238.26
10	Arts, Crafts & Sewing	1	144.23
11	Arts, Crafts & Sewing	2	106.64
12	Arts, Crafts & Sewing	3	240.67
13	Arts, Crafts & Sewing	4	186.33
14	Arts, Crafts & Sewing	6	267.35
15	Arts, Crafts & Sewing	7	378.37
16	Arts, Crafts & Sewing	9	39.52
17	Arts, Crafts & Sewing	10	350.92
18	Arts, Crafts & Sewing	11	50.28
19	Arts, Crafts & Sewing	12	150.38
20	Arts, Crafts & Sewing	14	96.27
21	Arts, Crafts & Sewing	15	150.38
22	Arts, Crafts & Sewing	16	346.35
23	Arts, Crafts & Sewing	17	414.87
24	Arts, Crafts & Sewing	20	294.86
25	Arts, Crafts & Sewing	[NULL]	3,455.68
26	Automotive	0	324.42
27	Automotive	1	418.6
28	Automotive	2	127.98
29	Automotive	3	157.55
30	Automotive	4	142.43
31	Automotive	6	62.17
32	Automotive	7	516.29

WHERE *tf.year* = 2020 AND *tf.month* BETWEEN 5 AND 11

Results 1 ×

SELECT *tf.year*, *tf.month*, *tf.city_category*, *tf.marital_status*, *SL* Enter a SQL expression to filter results (use Ctrl+Space)

	123 year	123 month	AZ city_category	123 marital_status	123 purchase_amount
1	2,020	5	A	0	1,382.2
2	2,020	6	A	0	454.1
3	2,020	7	A	0	996.33
4	2,020	8	A	0	751.14
5	2,020	9	A	0	478.33
6	2,020	10	A	0	1,277.83
7	2,020	11	A	0	160.86
8	2,020	[NULL]	A	0	5,500.79
9	2,020	5	A	1	697.76
10	2,020	6	A	1	528.4
11	2,020	7	A	1	418.43
12	2,020	8	A	1	661.9
13	2,020	9	A	1	458.85
14	2,020	10	A	1	148.07
15	2,020	11	A	1	539.4
16	2,020	[NULL]	A	1	3,452.81
17	2,020	[NULL]	A	[NULL]	8,953.6
18	2,020	5	B	0	1,139.56
19	2,020	6	B	0	524.27
20	2,020	7	B	0	762.74
21	2,020	8	B	0	853.2
22	2,020	9	B	0	749.27
23	2,020	10	B	0	990.11
24	2,020	11	B	0	936.07
25	2,020	[NULL]	B	0	5,955.22
26	2,020	5	B	1	340.16
27	2,020	6	B	1	378.28
28	2,020	7	B	1	549.45
29	2,020	8	B	1	308.82
30	2,020	9	B	1	358.04
31	2,020	10	B	1	1,629.7
32	2,020	11	B	1	770.06

Refresh Save Cancel Export data 200

Transaction_fact 1 ×

SELECT tf.stay_in_current_city_years, tf.gender, AVG(tf.price *) Enter a SQL expression to filter results

	123 stay_in_current_city_years	AZ gender	123 avg_purchase_amount
1	4	M	81.101167
2	3	M	78.813889
3	2	M	81.249977
4	3	F	75.519645
5	1	M	78.965248
6	4	F	82.856905
7	0	M	76.130836
8	1	F	75.148251
9	2	F	88.592787
10	0	F	88.717593

WHERE (SELECT COUNT(*))

Transaction_fact 1 ×

SELECT t1.product_category, t1.city_category, t1.purchase_amount Enter a SQL expression to filter results (use

	AZ product_category	AZ city_category	123 purchase_amount
1	Appliances	B	289.98
2	Appliances	A	89.93
3	Appliances	C	56.9
4	Arts, Crafts & Sewing	C	1,727.43
5	Arts, Crafts & Sewing	B	1,577.38
6	Arts, Crafts & Sewing	A	1,547.2
7	Automotive	A	1,569.38
8	Automotive	B	1,534.89
9	Automotive	C	901.53
10	Baby	B	1,378.55
11	Baby	C	807.74
12	Baby	A	504.03
13	Books, Movies & Music	A	846.48
14	Books, Movies & Music	B	656.95
15	Books, Movies & Music	C	499.92
16	Clothing	B	1,295.62
17	Clothing	C	1,224.61
18	Clothing	A	1,219.66
19	EEEE	B	211.5
20	EEEE	C	186.12
21	EEEE	A	143.82
22	Electronics	B	5,539.01
23	Electronics	A	4,378.9
24	Electronics	C	4,125.47
25	Furniture	A	1,108.78
26	Furniture	C	832.17
27	Furniture	B	750.57
28	Grocery	B	30,539.77
29	Grocery	C	29,096.51
30	Grocery	A	24,593.3
31	Health & Beauty	B	32,114.96

Refresh Save Cancel Export

Transaction_fact 1 x

SELECT cur.product_category, cur.month AS current_month, 123 current_revenue, 123 previous_revenue, 123 growth_percent

Enter a SQL expression to filter results (use Ctrl+Space)

	product_category	current_month	current_revenue	previous_revenue	growth_percent
1	Appliances	3	143.52	[NULL]	[NULL]
2	Appliances	12	24.32	[NULL]	[NULL]
3	Arts, Crafts & Sewing	1	60.81	[NULL]	[NULL]
4	Arts, Crafts & Sewing	2	169.74	60.81	179.131722
5	Arts, Crafts & Sewing	3	185.91	169.74	9.526334
6	Arts, Crafts & Sewing	4	150.25	185.91	-19.181324
7	Arts, Crafts & Sewing	7	56.58	[NULL]	[NULL]
8	Arts, Crafts & Sewing	8	201.32	56.58	255.814776
9	Arts, Crafts & Sewing	10	124.22	[NULL]	[NULL]
10	Arts, Crafts & Sewing	11	150.38	124.22	21.059411
11	Arts, Crafts & Sewing	12	133.43	150.38	-11.271446
12	Automotive	4	173.26	[NULL]	[NULL]
13	Automotive	5	88.36	173.26	-49.001501
14	Automotive	7	256.57	[NULL]	[NULL]
15	Automotive	9	57.92	[NULL]	[NULL]
16	Automotive	12	68.22	[NULL]	[NULL]
17	Baby	3	20.56	[NULL]	[NULL]
18	Baby	6	42.55	[NULL]	[NULL]
19	Baby	8	155.48	[NULL]	[NULL]
20	Baby	9	67.22	155.48	-56.766144
21	Baby	10	133.81	67.22	99.062779
22	Books, Movies & Music	2	141.08	[NULL]	[NULL]
23	Books, Movies & Music	4	211.62	[NULL]	[NULL]
24	Clothing	2	127.31	[NULL]	[NULL]
25	Clothing	3	11.26	127.31	-91.155447
26	Clothing	7	73.05	[NULL]	[NULL]
27	Clothing	9	85.59	[NULL]	[NULL]
28	Clothing	11	134.48	[NULL]	[NULL]
29	Clothing	12	98.72	134.48	-26.591315
30	EEEE	1	16.92	[NULL]	[NULL]
31	EEEE	1	8.16	[NULL]	[NULL]

[illegible]

Results 1 ×

(SELECT tf.product_id, tf.month, SUM(tf.price * tf.quantity) AS revenue FROM tensorflow tf)

Grid

Text

	product_id	month	revenue	day_type
1	1,642	1	239.79	Weekend
2	1,945	7	234.15	Weekend
3	1,443	8	231.06	Weekend
4	1,121	9	228.87	Weekend
5	477	1	228.36	Weekend
6	639	5	465.01	Weekday
7	99	6	366.3	Weekday
8	1,104	10	271.4	Weekday
9	1,366	4	238.23	Weekday
10	23	6	236.28	Weekday

Transaction_fact 1 x

SELECT curr_store_id, curr_quarter AS current_quarter, curr_rev Enter a SQL expression to filter results (use Ctrl+Space)

	123 store_id	123 current_quarter	123 current_revenue	123 previous_revenue	123 growth_percent
1	1		1,120.56	[NULL]	[NULL]
2	1		1,202.95	1,120.56	7.35
3	1		1,822.85	1,202.95	51.53
4	1		1,925.81	1,822.85	5.65
5	2		1,989.21	[NULL]	[NULL]
6	2		2,453.01	1,989.21	23.32
7	2		1,723.57	2,453.01	-29.74
8	2		897.5	1,723.57	-47.93
9	3		4,473.66	[NULL]	[NULL]
10	3		2,489.4	4,473.66	-44.35
11	3		2,543.91	2,489.4	2.19
12	3		3,687.25	2,543.91	44.94
13	4		1,750.32	[NULL]	[NULL]
14	4		1,427.53	1,750.32	-18.44
15	4		1,362.61	1,427.53	-4.55
16	4		1,175.92	1,362.61	-13.7
17	5		4,036.52	[NULL]	[NULL]
18	5		3,658.13	4,036.52	-9.37
19	5		3,940.33	3,658.13	7.71
20	5		4,686.12	3,940.33	18.93
21	6		1,477.82	[NULL]	[NULL]
22	6		1,892.92	1,477.82	28.09
23	6		2,282.71	1,892.92	20.59
24	6		1,883.63	2,282.71	-17.48
25	7		631.82	[NULL]	[NULL]
26	7		721.94	631.82	14.26
27	7		915.67	721.94	26.83
28	7		942.11	915.67	2.89
29	8		272.28	[NULL]	[NULL]
30	8		222.21	272.28	-18.39
31	8		357.35	222.21	60.82
32	8		377.56	357.35	5.66

Refresh Save Cancel Export data 200

Transaction_fact 1 ×

SELECT tf.store_id, tf.supplier_id, tf.product_id, SUM(tf.price) Enter a SQL expression to filter results (use Ctrl+Space)

	123 store_id	123 supplier_id	123 product_id	123 total_sales
1	1	1	849	2,043.16
2	1	1	155	1,479.15
3	1	1	385	1,268.28
4	1	1	696	1,211.84
5	1	1	120	1,104.15
6	1	1	53	1,057.29
7	1	1	979	1,034.6
8	1	1	273	986.7
9	1	1	821	887.36
10	1	1	949	851.07
11	1	1	312	689.08
12	1	1	463	679.92
13	1	1	1,281	645.54
14	1	1	999	638.85
15	1	1	785	637.67
16	1	1	1,039	632.1
17	1	1	871	573.04
18	1	1	719	539.89
19	1	1	1,272	529.23
20	1	1	659	491.04
21	1	1	632	480.92
22	1	1	1,107	480.2
23	1	1	577	475.3
24	1	1	1,243	470.46
25	1	1	326	469.14
26	1	1	423	463.45
27	1	1	167	459.14
28	1	1	440	450.5
29	1	1	1,683	448.98
30	1	1	1,103	442.32
31	1	1	39	428.74

Refresh Save Cancel Export data 200

Grid		123 product_id	A-Z season	123 total_sales
		[NULL]	[NULL]	73,282.2
Text	1	[NULL]	[NULL]	73,282.2
	2	5	[NULL]	69.42
	3	5	Winter	69.42
	4	8	[NULL]	88.05
	5	8	Fall	88.05
	6	10	[NULL]	122.13
	7	10	Summer	81.42
	8	10	Winter	40.71
	9	11	[NULL]	649.8
	10	11	Fall	324.9
	11	11	Summer	162.45
	12	11	Winter	162.45
	13	14	[NULL]	138.72
	14	14	Fall	34.68
	15	14	Spring	17.34
	16	14	Summer	52.02
	17	14	Winter	34.68
	18	20	[NULL]	585.04
	19	20	Spring	219.39
	20	20	Summer	219.39
	21	20	Winter	146.26
	22	21	[NULL]	144.24
	23	21	Spring	144.24
	24	23	[NULL]	236.28
	25	23	Summer	236.28
	26	26	[NULL]	163.35
	27	26	Winter	163.35
	28	28	[NULL]	155.76
	29	28	Fall	155.76
	30	31	[NULL]	101.46
	31	31	Winter	101.46
	32	36	[NULL]	16.68
	33	36	Winter	16.68
	34	40	[NULL]	154.25
Record				

Refresh
Save
Cancel

15

	123 store_id	123 supplier_id	123 current_month	123 current_revenue	123 previous_revenue	123 volatility
1	1	1	1	645.43	[NULL]	[NULL]
2	1	1	2	411.7	645.43	-36.21
3	1	1	3	848.93	411.7	106.2
4	1	1	4	403.7	848.93	-52.45
5	1	1	5	188.61	403.7	-53.28
6	1	1	6	592.65	188.61	214.22
7	1	1	7	265.96	592.65	-55.12
8	1	1	8	489.27	265.96	83.96
9	1	1	9	731.64	489.27	49.54
10	1	1	10	738.38	731.64	0.92
11	1	1	11	540.89	738.38	-26.75
12	1	1	12	350.35	540.89	-35.23
13	2	2	1	920.81	[NULL]	[NULL]
14	2	2	2	460.57	920.81	-49.98
15	2	2	3	470.5	460.57	2.16
16	2	2	4	356	470.5	-24.34
17	2	2	5	598.82	356	68.21
18	2	2	6	106.53	598.82	-82.21
19	2	2	7	746.07	106.53	600.34
20	2	2	8	429.38	746.07	-42.45
21	2	2	9	765.66	429.38	78.32
22	2	2	10	629.15	765.66	-17.83
23	2	2	11	470.86	629.15	-25.16
24	2	2	12	158.37	470.86	-66.37
25	3	3	1	1,092.32	[NULL]	[NULL]
26	3	3	2	1,086.76	1,092.32	-0.51
27	3	3	3	861.17	1,086.76	-20.76
28	3	3	4	832.06	861.17	-3.38
29	3	3	5	1,736.29	832.06	108.67
30	3	3	6	852.12	1,736.29	-50.92
31	3	3	7	1,803.28	852.12	111.62
32	3	3	8	2,407.11	1,803.28	33.49
33	3	3	9	978.5	2,407.11	-59.35
34	3	3	10	2,211.37	978.5	126

Refresh Save Cancel Export data 200 93 93 row(s) fetched

16

Transaction_fact 1 ×			
SELECT t1.product_id AS product_1, t2.product_id AS product_2, t1.times_bought AS times_bought_together			
	123 product_1	123 product_2	123 times_bought_together
1	223	223	40
2	95	95	39
3	352	352	35
4	155	155	34
5	347	347	32

Results 1 ×

SELECT store_id, supplier_id, product_id, SUM(price * quantity) Enter a SQL expression to filter results (u

	123 store_id	123 supplier_id	123 product_id	123 yearly_revenue
1	1	1	[NULL]	8,365.4
2	1	1	53	243.99
3	1	1	56	32.38
4	1	1	83	168.14
5	1	1	91	200.45
6	1	1	120	714.45
7	1	1	143	61.56
8	1	1	155	98.61
9	1	1	167	41.74
10	1	1	184	54.75
11	1	1	219	53.52
12	1	1	245	207.9
13	1	1	273	131.56
14	1	1	295	54.25
15	1	1	309	24.12
16	1	1	312	196.88
17	1	1	347	20.61
18	1	1	376	41.77
19	1	1	401	28.8
20	1	1	423	224.25
21	1	1	463	113.32
22	1	1	504	140.45
23	1	1	529	78.76
24	1	1	539	233.19
25	1	1	565	27.24
26	1	1	577	95.06
27	1	1	582	7.74
28	1	1	595	47.5
29	1	1	603	37.62
30	1	1	632	174.88
31	1	1	646	63.69
32	1	1	659	79.2
33	1	1	696	151.48
34	1	1	710	178.16

Refresh Save Cancel Export

Transaction_fact 1 ×

SELECT product_id, SUM(CASE WHEN month BETWEEN 1 AND 12 THEN revenue ELSE 0 END) AS yearly_revenue, SUM(CASE WHEN month BETWEEN 1 AND 12 THEN quantity ELSE 0 END) AS yearly_quantity

	product_id	123 revenue_h1	123 quantity_h1	123 revenue_h2	123 quantity_h2	123 yearly_revenue	123 yearly_quantity
1	49	359.3	5	503.02	7	862.32	12
2	377	405.84	6	405.84	6	811.68	12
3	120	454.65	7	259.8	4	714.45	11
4	23	472.56	6	236.28	3	708.84	9
5	259	128	2	576	9	704	11
6	477	304.48	4	380.6	5	685.08	9
7	11	162.45	3	487.35	9	649.8	12
8	382	174.99	3	466.64	8	641.63	11
9	1,149	110.5	2	497.25	9	607.75	11
10	277	148.96	2	446.88	6	595.84	8
11	20	365.65	5	219.39	3	585.04	8
12	1,170	217.77	3	362.95	5	580.72	8
13	901	289.08	4	289.08	4	578.16	8
14	841	0	0	550.08	9	550.08	9
15	26	326.7	6	217.8	4	544.5	10
16	47	226.85	5	317.59	7	544.44	12
17	1,443	308.08	4	231.06	3	539.14	7
18	1,062	200.61	3	334.35	5	534.96	8
19	137	534.72	8	0	0	534.72	8
20	571	470.16	8	58.77	1	528.93	9
21	730	293.56	4	220.17	3	513.73	7
22	99	366.3	5	146.52	2	512.82	7
23	881	287.92	4	215.94	3	503.86	7
24	105	117.68	2	353.04	6	470.72	8
25	520	313.8	6	156.9	3	470.7	9
26	639	465.01	7	0	0	465.01	7
27	1,303	126.3	3	336.8	8	463.1	11
28	647	383.7	5	76.74	1	460.44	6
29	71	152.8	2	305.6	4	458.4	6
30	109	65.28	1	391.68	6	456.96	7
31	825	152.04	2	304.08	4	456.12	6
32	614	226.5	3	226.5	3	453	6
33	417	190.65	3	254.2	4	444.85	7

Refresh Save Cancel Export data 200 200+ 200 row(s) fetched - 0.010s (0.002s fetch), on 2025

Transaction_fact 1 ×

WITH daily_sales AS (SELECT product_id, date, SUM(price * qty) AS daily_revenue, AVG(price * qty) AS avg_revenue FROM Transaction_fact 1 WHERE status = 'Normal')

	product_id	date	daily_revenue	avg_revenue	status
1	5	2015-02-28	34.71	52.065	Normal
2	5	2016-03-06	34.71	52.065	Normal
3	5	2017-10-25	104.13	52.065	Normal
4	5	2018-03-06	34.71	52.065	Normal
5	5	2018-07-11	34.71	52.065	Normal
6	5	2020-02-18	69.42	52.065	Normal
7	8	2015-04-23	58.7	67.085714	Normal
8	8	2015-08-11	88.05	67.085714	Normal
9	8	2015-10-19	88.05	67.085714	Normal
10	8	2016-09-08	29.35	67.085714	Normal
11	8	2016-10-25	58.7	67.085714	Normal
12	8	2017-03-23	58.7	67.085714	Normal
13	8	2017-07-03	58.7	67.085714	Normal
14	8	2017-08-07	29.35	67.085714	Normal
15	8	2017-12-10	88.05	67.085714	Normal
16	8	2019-01-02	88.05	67.085714	Normal
17	8	2019-04-28	88.05	67.085714	Normal
18	8	2019-07-07	29.35	67.085714	Normal
19	8	2019-07-13	88.05	67.085714	Normal
20	8	2020-10-26	88.05	67.085714	Normal
21	10	2015-01-17	81.42	84.6768	Normal
22	10	2015-08-30	81.42	84.6768	Normal
23	10	2016-03-05	81.42	84.6768	Normal
24	10	2016-03-10	81.42	84.6768	Normal
25	10	2016-04-13	40.71	84.6768	Normal
26	10	2016-08-25	122.13	84.6768	Normal
27	10	2016-11-16	81.42	84.6768	Normal
28	10	2016-12-28	40.71	84.6768	Normal
29	10	2016-12-31	81.42	84.6768	Normal
30	10	2017-03-01	81.42	84.6768	Normal
31	10	2017-03-16	81.42	84.6768	Normal
32	10	2017-03-26	40.71	84.6768	Normal
33	10	2017-04-24	81.42	84.6768	Normal

Refresh Save Cancel Filter Sort Group Hide Columns Export data 200

Transaction_fact(+) 1 x				
SELECT * FROM db.STORE_QUARTERLY_SALES ORDER BY store Enter a SQL expression to filter results (use Ctrl+Space)				
Grid	123 store_id	Az q	123 quarterly_sales	
Text	1	Q1	14,557.57	
	2	Q2	12,758.63	
	3	Q3	14,845.71	
	4	Q4	15,202.22	
	5	Q1	11,161.8	
	6	Q2	9,852.41	
	7	Q3	10,055.2	
	8	Q4	7,800.61	
	9	Q1	29,104.21	
	10	Q2	29,249.21	
	11	Q3	27,797.61	
	12	Q4	30,481.11	
	13	Q1	16,606.41	
	14	Q2	18,106.02	
	15	Q3	14,906.02	
	16	Q4	15,994.16	
	17	Q1	39,069.75	
	18	Q2	39,719.1	
	19	Q3	40,559.05	
	20	Q4	41,870.66	
	21	Q1	17,623.39	
	22	Q2	16,912	
	23	Q3	16,982.65	
	24	Q4	16,411.87	
	25	Q1	7,284.05	
	26	Q2	5,847.22	
	27	Q3	7,355.91	
	28	Q4	6,385.31	
	29	Q1	2,059.14	
	30	Q2	2,315.12	
	31	Q3	2,041.13	
	32	Q4	2,398.92	
Record				
Refresh Save Cancel				
Export data 200				